

CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan

w/ Ram Kesavan

Lecture 9-10: Key-Value/NoSQL Stores

Ack: Indranil Gupta

Announcements

- HW2 Released, due on Sunday 10/4 in less than 2 weeks (start early, no extensions!)
- MP2 is already out, due soon 9/27 (no extensions!)
- MP1 grades released (see Piazza post)
- HW1 being graded
- In class midterm October 8th
 - Make sure you're in class that day!
 - There will be two rooms, different buildings (will update on Piazza and course webpage soon)



The Key-value Abstraction

- (Business) Key $\rightarrow$ Value
- (twitter.com) tweet id $\rightarrow$ information about tweet
- (amazon.com) item number $\rightarrow$ information about it
- (kayak.com) Flight number $\rightarrow$ information about flight, e.g., availability
- (yourbank.com) Account number $\rightarrow$ information about it

The Key-value Abstraction (2)

- It's a dictionary datastructure.
 - Insert, lookup, and delete by key
 - E.g., hash table, binary tree
- But distributed.
- Sound familiar? Remember Distributed Hash tables (DHT) in P2P systems?
- It's not surprising that key-value stores reuse many techniques from DHTs.

Isn't that just a database?

- Yes, sort of
- Relational Database Management Systems (RDBMSs) have been around for ages
 - E.g., MySQL
- Schema-based, i.e., structured tables
- Each row (data item) in a table has a primary key that is unique within that table
- Queried using SQL (Structured Query Language)
- Supports joins

Relational Database Example

users table

user_id	name	zipcode	blog_url	blog_id
101	Alice	12345	alice.net	1
422	Charlie	45783	charlie.com	3
555	Bob	99910	bob.blogspot.com	2

↑
Primary keys

↑
Foreign keys

blog table

id	url	last_updated	num_posts
1	alice.net	5/2/14	332
2	bob.blogspot.com	4/2/13	10003
3	charlie.com	6/15/14	7

Example SQL queries

1. `SELECT zipcode
FROM users
WHERE name = "Bob"`
2. `SELECT url
FROM blog
WHERE id = 3`
3. `SELECT users.zipcode, blog.num_posts
FROM users JOIN blog
ON users.blog_url = blog.url`

users.zipcode	blog.num_posts
12345	332
45783	7
99910	10003

Mismatch with today's workloads

- Data: Large and unstructured
- Lots of random reads and writes
- Sometimes write-heavy
- Foreign keys rarely needed
- Joins infrequent

Needs of Today's Workloads

- Speed
- Avoid Single point of Failure (SPoF)
- Low TCO (Total cost of operation/ownership)
- Fewer system administrators
- Incremental Scalability
- Scale out, not up
 - What?

Scale out, not Scale up

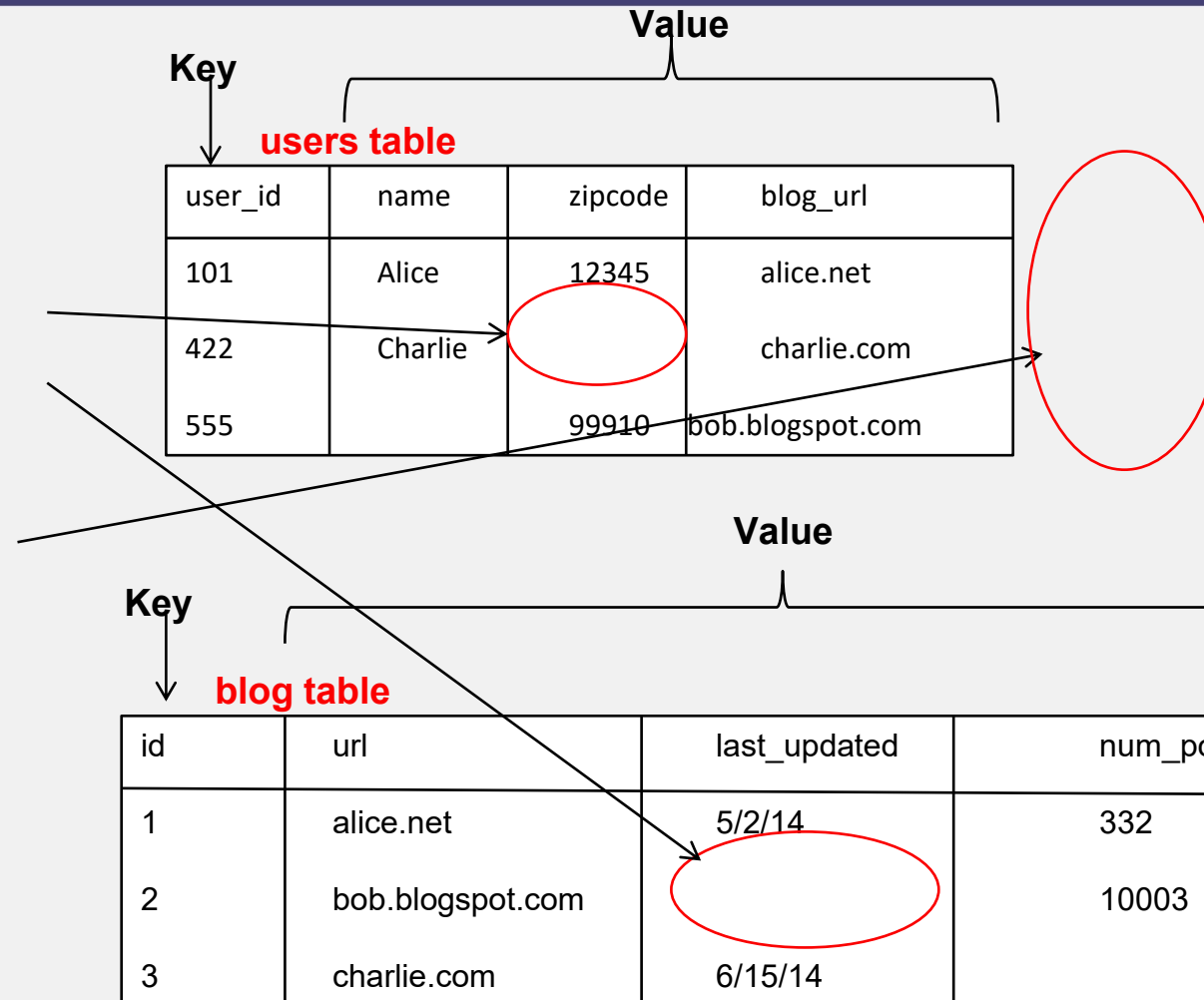
- Scale up = grow your cluster capacity by replacing with more powerful machines
 - Traditional approach
 - Not cost-effective, as you're buying above the sweet spot on the price curve
 - And you need to replace machines often
- Scale out = incrementally grow your cluster capacity by adding more COTS machines (Components Off the Shelf)
 - Cheaper
 - Over a long duration, phase in a few newer (faster) machines as you phase out a few older machines
 - Used by most companies who run datacenters and clouds today

Key-value/NoSQL Data Model

- NoSQL = “Not Only SQL”
- Necessary API operations: `get(key)` and `put(key, value)`
 - And some extended operations, e.g., “CQL” in Cassandra key-value store
- Tables
 - “Column families” in Cassandra, “Table” in HBase, “Collection” in MongoDB
 - but ...
 - May be unstructured: May not have schemas
 - Some columns may be missing from some rows
 - Don’t always support joins or have foreign keys
 - Can have index tables, just like RDBMSs

Key-value/NoSQL Data Model

- Unstructured
- Columns Missing from some Rows
- No schema imposed
- No foreign keys, joins may not be supported



CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan

w/ Ram Kesavan

Lecture 9-10: Key-Value/NoSQL Stores

Ack: Indranil Gupta

Some history...

Earlier: RDBMS

- SQL, ACID transactions, enterprise workloads
- Systems: Oracle, DB2, SQL Server, MySQL

~late '00s and '10s: Rise of NoSQL

- Large web companies build distributed stores to handle massive scale
- Focus on partitioning, replication, high availability, and simpler data models
- Systems: Google Bigtable, Amazon Dynamo, **Cassandra, HBase, MongoDB**

Today: Converging

- SQL databases increasingly support horizontal scale-out (cloud databases)
- NoSQL databases increasingly add transactions and richer query capabilities
- Systems: Spanner, CockroachDB, distributed PostgreSQL, DynamoDB

Agenda

Design of a real key-value store, Cassandra.

- Replication

- Consistency vs. Availability tradeoff

- Quorums

- Local storage

HBase, another KV store and Consistency models (subsequent lectures)

Cassandra

- A distributed key-value store
- Intended to run in a **datacenter** (and also across DCs)
- Originally designed at Facebook (influenced by Amazon Dynamo [1])
- Open-sourced later, today an Apache project [2]
- Widely used in production
 - IBM, Adobe, HP, eBay, Ericsson, Symantec, Twitter, Spotify, PBS Kids, Netflix
 - Netflix: uses Cassandra to keep track of your current position in the video you're watching
- (Version from 2015)

[1] <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>

[2] <https://github.com/apache/cassandra>

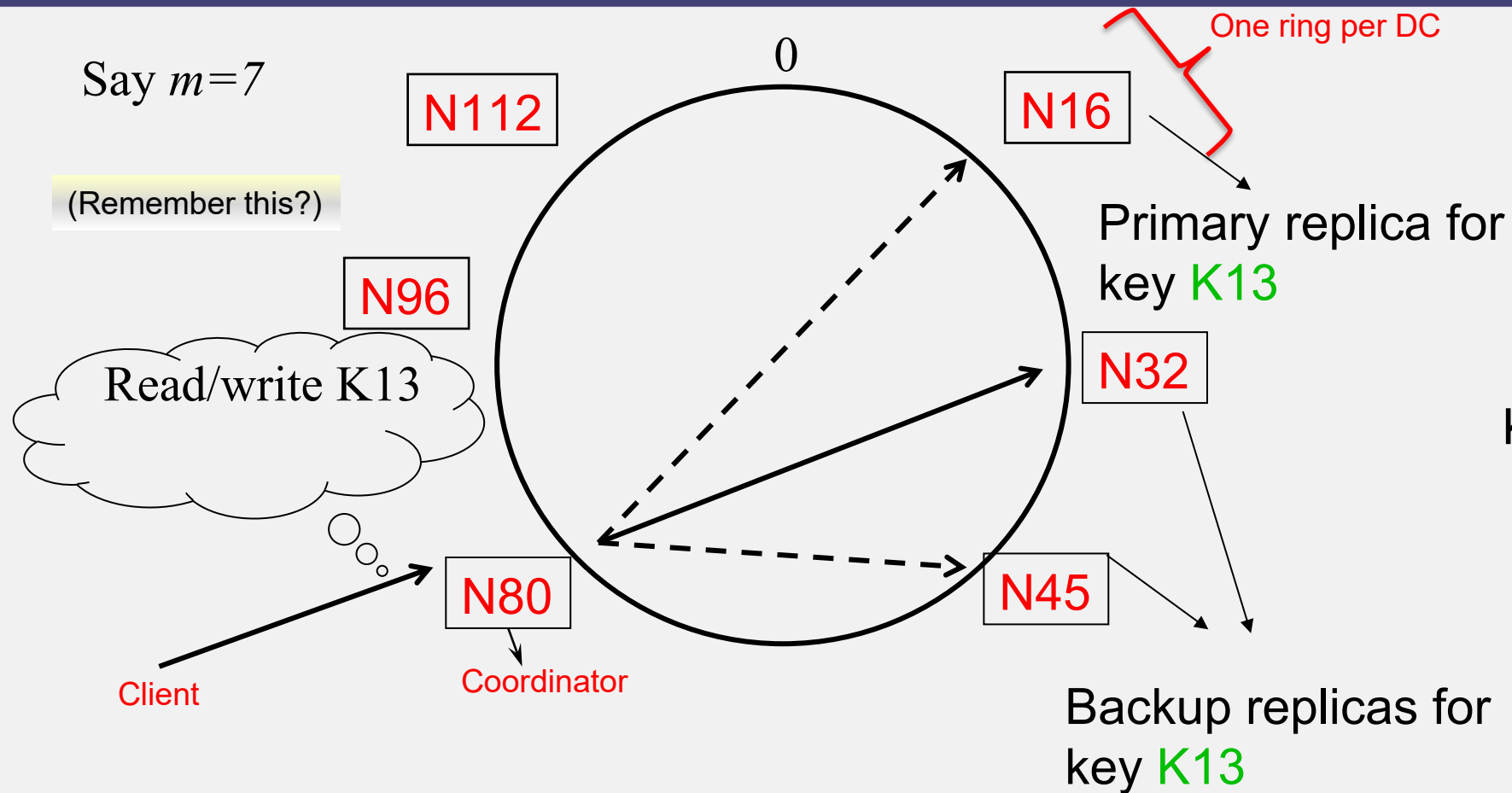
Let's go Inside Cassandra:

Key -> Server Mapping

- How do you decide which server(s) a key-value resides on?

? Why okay to store full list at all nodes in Cassandra but not Chord ?

? Routing hops?



Key replicated on its r successors
 r -replication factor

Cassandra uses a Ring-based DHT but without finger tables
Each node has the full membership list

Data Placement Strategies

Replication Strategy: two options:

1. SimpleStrategy: uses the Partitioner, of which there are two kinds
 1. *RandomPartitioner*: Chord-like hash partitioning
 2. *ByteOrderedPartitioner*: Assigns ranges of keys to servers.
 - Easier for range queries (e.g., Get me all twitter users starting with [a-b])
2. NetworkTopologyStrategy:
 - E.g., Two replicas per DC
 - Per DC
 - First replica placed according to Partitioner
 - Then go clockwise around ring until you hit a different rack
 - Cassandra uses a component called **snitches** to map IPs to racks and DCs

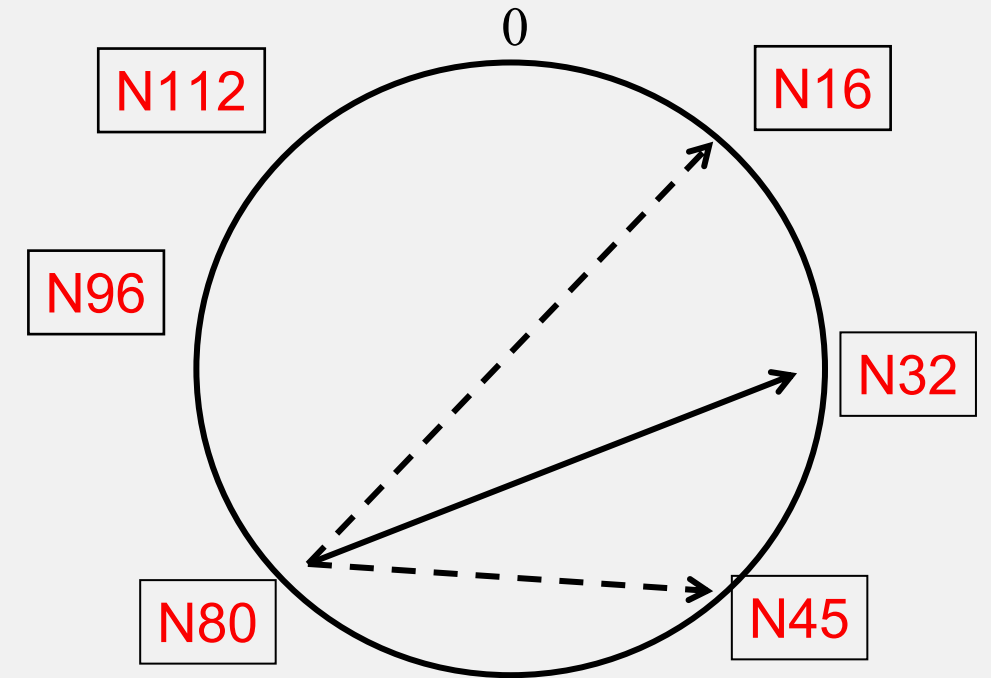
Writes

- Client sends write to one coordinator node in Cassandra cluster
 - Coordinator may be per-key, or per-client, or per-query
 - Typically, the successor of key in the ring
- Coordinator assigns its current timestamp for the write (or client supplies it)
 - Per-key Coordinator ensures writes for the key are ordered
- Coordinator sends write to all replica nodes responsible for key (let's call it **N** from now)
- A replica records the write to its *local storage* – We'll see later.
- When first **X** replicas respond, coordinator acknowledges client
 - *X*? We'll see later.
- Note that multiple writes to same key **K** can be in progress at the same time.

Writes – Hinted Handoff

Always writable: Hinted Handoff mechanism

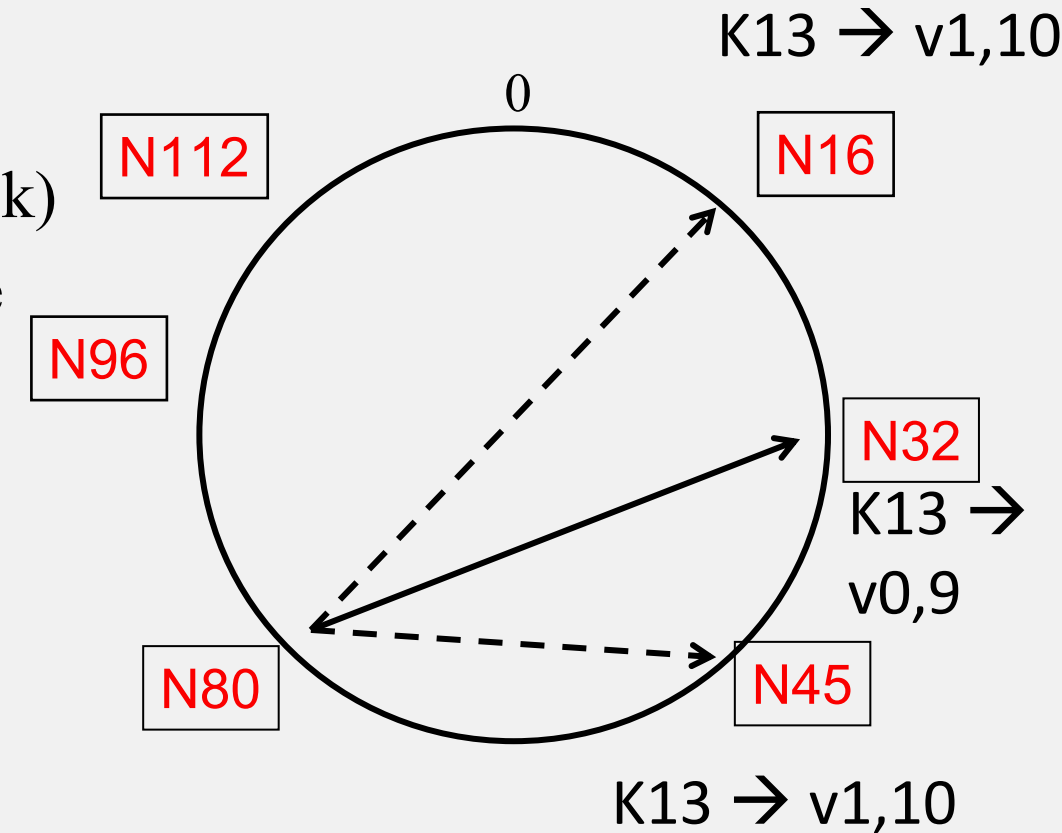
- If any replica is down, the coordinator writes to all other replicas
 - And keeps the write locally until down replica comes back up.
 - Catch up replica after it is up
-
- When all replicas are down, the Coordinator buffers writes (for up to a few hours).



Reads

Read: Similar to writes

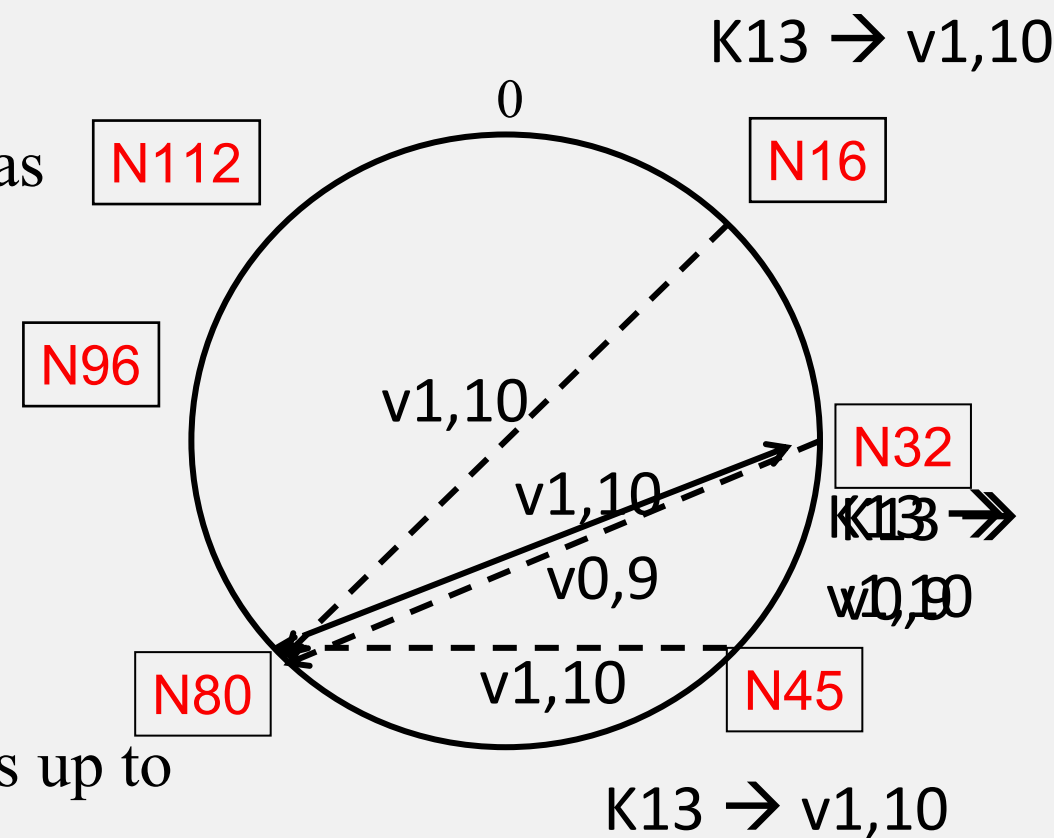
- Coordinator can contact X replicas (e.g., in same rack)
 - Coordinator sends read to replicas that have responded quickest in past
 - Or, send to all N and wait for first X
- When X replicas respond, coordinator returns the **latest-timestamped** value from among those X
 - (X? We'll see later.)



Reads – read repair

After X responses, in the **background**

- Coordinator also fetches value from other replicas
 - Checks consistency
 - Initiates a **read repair** if any two values are different
 - To install the latest value
- This mechanism seeks to eventually bring all replicas up to date



Membership

- Any server in cluster could be the coordinator
- Every server maintains a full list of all other servers
 - updated automatically as servers join, leave, fail
- Cassandra uses gossip-based cluster membership
- One ring per datacenter
 - Per-DC leader elected to coordinate with other DCs
 - Election done via Zookeeper, which runs a Paxos (consensus) variant (Paxos: elsewhere in this course)
 - Leader tells new nodes what key ranges they are replicas for

Failure detection

- Use heartbeat-style Failure Detector (uses Φ Accrual detector)
 - Adaptively sets timeout based on underlying network behavior
 - Entry not deleted on failure (unlike fail-stop model)
 - Cassandra uses fail-recover model and so nodes just marked down/up

Mystery of “X”: CAP Theorem

- Proposed by Eric Brewer (Berkeley)
- Subsequently proved by Gilbert and Lynch (NUS and MIT)
- In a distributed system you can satisfy at most 2 out of the 3 guarantees:
 1. **Consistency**: all nodes see same data at any time, or reads return latest written value by any client
 2. **Availability**: the system allows operations all the time, and operations return quickly
 3. **Partition-tolerance**: the system continues to work in spite of network partitions

Why is Availability Important?

- Availability = Reads/writes complete reliably and quickly.
- Measurements have shown that a 500 ms increase in latency for operations at Amazon.com or at Google.com can cause a 20% drop in revenue.
 - More recent measurements: Amazon: 100 ms increase → 1% drop in sales. Akamai: 100 ms → 7% drop.
- At Amazon, each added millisecond of latency implies a \$6M yearly loss.
 - More recent measurements: Amazon: 1 s → \$1.6 B.
- User cognitive drift: If more than a second elapses between clicking and material appearing, the user's mind is already somewhere else
- SLAs (Service Level Agreements) written by providers predominantly deal with latencies faced by clients.

Why is Consistency Important?

- Consistency = all nodes see same data at any time, or reads return latest written value by any client.
- When you access your bank or investment account via multiple clients (laptop, workstation, phone, tablet), you want the updates done from one client to be visible to other clients.
- When thousands of customers are looking to book a flight, all updates from any client (e.g., book a flight) should be accessible by other clients.

Why is Partition-Tolerance Important?

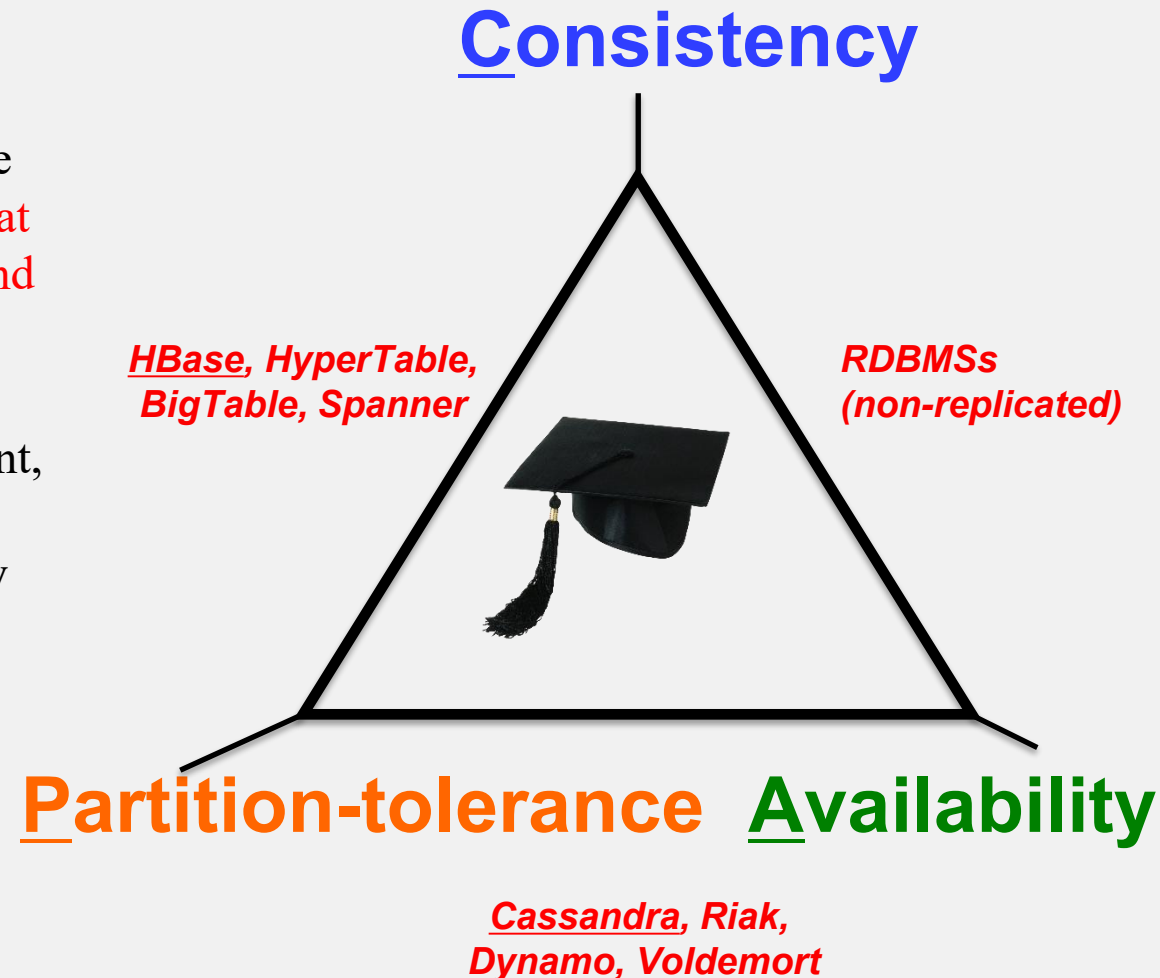
- Partitions can happen across datacenters when the Internet gets disconnected
 - Internet router outages
 - Under-sea cables cut
 - DNS not working
- Partitions can also occur within a datacenter, e.g., a rack switch outage
- Still desire system to continue functioning normally under this scenario

CAP Theorem Fallout

- Since partition-tolerance is essential in today's cloud computing systems, CAP theorem implies that a system has to choose between consistency and availability
- Cassandra
 - Eventual (weak) consistency, Availability, Partition-tolerance
- Traditional RDBMSs
 - Strong consistency over availability under a partition

CAP Tradeoff

- Starting point for NoSQL Revolution
- A distributed storage system can achieve **at most two of C, A, and P**.
- When partition-tolerance is important, you have to choose between consistency and availability



Eventual Consistency



What mechanisms in Cassandra we saw earlier helps with eventual convergence?

- If all writes stop (to a key), then all its values (replicas) will converge eventually.
- If writes continue, then system always tries to keep converging.
 - Moving “wave” of updated values lagging behind the latest values sent by clients, but always trying to catch up.
- May still return stale values to clients (e.g., if many back-to-back writes).
- But works well when there a few periods of low writes – system converges quickly.

RDBMS vs. Key-value stores

- While RDBMS provide **ACID**
 - Atomicity
 - Consistency
 - Isolation
 - Durability
- Key-value stores like Cassandra provide **BASE**
 - Basically Available Soft-state Eventual Consistency
 - Prefers Availability over Consistency

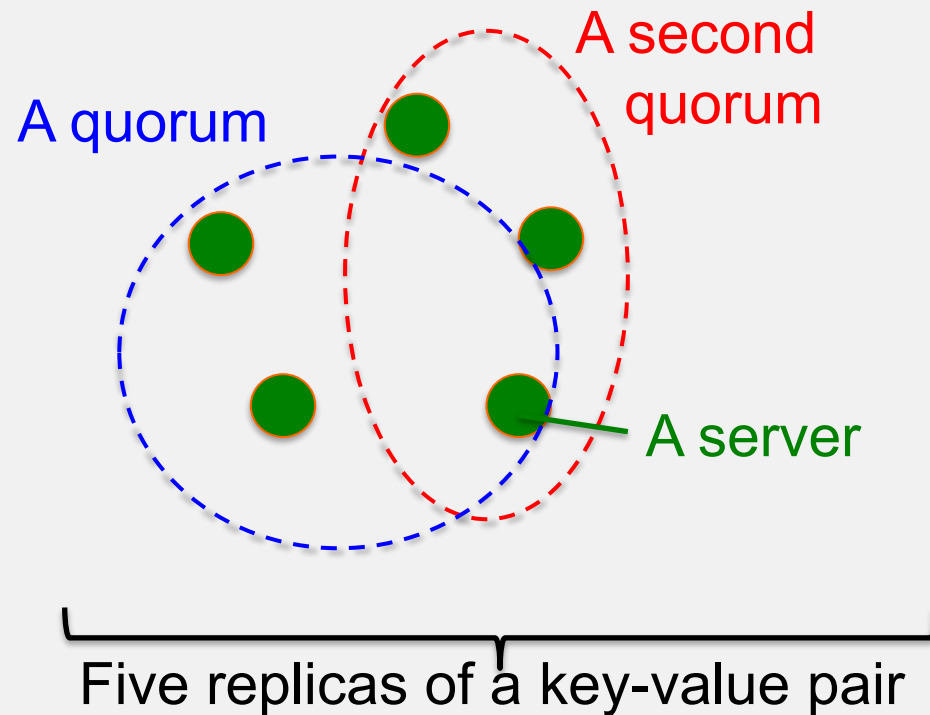
Back to Cassandra: Mystery of X

- Cassandra has [consistency levels](#)
- Client is allowed to choose a consistency level for each operation (read/write)
 - ANY: any server (may not be replica)
 - Fastest: coordinator caches write and replies quickly to client
 - ALL: all replicas
 - Ensures strong consistency, but slowest
 - ONE: at least one replica
 - Faster than ALL
 - QUORUM: quorum across all replicas in all datacenters (DCs)
 - What?

Quorums?

In a nutshell:

- Quorum = majority
 - $> 50\%$
- **Any two quorums intersect** no matter how you select the quorums
 - Client 1 does a write in red quorum
 - Then client 2 does read in blue quorum
- At least one server in blue quorum returns latest write
- Quorums faster than ALL, but still ensure strong consistency



Quorums in Detail

- Several key-value/NoSQL stores (e.g., Riak and Cassandra) use quorums.
- Reads
 - Client specifies value of **R** ($\leq N$ = total number of replicas of that key).
 - R = read consistency level.
 - Coordinator waits for R replicas to respond before sending result to client (returns the latest-timestamped value)
 - In background, coordinator checks for consistency of remaining $(N-R)$ replicas, and initiates read repair if needed.

Quorums in Detail (Contd.)

- Writes
- Client specifies W ($\leq N$)
 - W = write consistency level.
 - Coordinator writes new value to W replicas and returns.
 - Coordinator blocks until W replicas perform the write

Quorums in Detail (Contd.)



Does the system choose consistency or availability upon partitions?

1. $W=N, R=1$
2. $W=R=QUORUM$
3. $W=R=ANY$

- R = read replica count, W = write replica count
- Two necessary conditions for consistency:
 1. $W+R > N$
 2. $W > N/2$
- Select values based on application
 - $(W=1, R=1)$: very few writes and reads
 - $(W=N, R=1)$: great for read-heavy workloads
 - $(W=N/2+1, R=N/2+1)$: great for write-heavy workloads
 - $(W=1, R=N)$: great for write-heavy workloads with mostly one client writing per key

Cassandra Consistency Levels (Contd.)

- Client is allowed to choose a consistency level for each operation (read/write)
 - ANY: any server (may not be replica)
 - Fastest: coordinator may cache write and reply quickly to client
 - ALL: all replicas
 - Slowest, but ensures strong consistency
 - ONE: at least one replica
 - Faster than ALL
 - QUORUM: quorum across all replicas in all datacenters (DCs)
 - Global consistency, but still fast
 - LOCAL_QUORUM: quorum in coordinator's DC
 - Faster: only waits for quorum in first DC client contacts
 - EACH_QUORUM: quorum in every DC
 - Lets each DC do its own quorum: supports hierarchical replies

Announcements

- (All) HW2 due 10/4 @ 11:59 pm (Sunday!) US Central (hard deadline)
- (All) Midterm Thursday (10/8): Written, in class
- (4cr) **MP2 Demo, Signup, and Submission instructions release on Piazza**
 - ✓ (Due by Fri) [MP Demo Signup Sheet]
 - ✓ (Due by Sun) [a. MP Submission Form & push MP code to GitHub]
 - ✓ (Due by Sun 13) [MP Report Submission on Gradescope]
 - ✓ (On Mon) MP Demos

Local storage at a replica – Writes



How slow is a disk seek compared to DRAM?

- a. 5x b. 50x
- c. 5000x d. 50000x

Need to be lock-free and fast (no reads or disk seeks)

On receiving a write

1. Log it in disk commit log (for failure recovery)

Writes to a data structure called Log-Structured Merge-tree (LSM)

2. Make changes to appropriate memtables

- **Memtable** = In-memory representation of multiple key-value pairs
- Cache that can be searched by key
- Write-back cache as opposed to write-through

LSM animation

Writes at a replica node

On receiving a write

1. Log it in disk commit log (for failure recovery)
2. Make changes to appropriate memtables
 - **Memtable** = In-memory representation of multiple key-value pairs
 - Cache that can be searched by key
 - Write-back cache as opposed to write-through

Later, when memtable is full or old, flush to disk

- Data File: An **SSTable** (Sorted String Table) – list of key-value pairs, sorted by key
- *SSTables are immutable (once created, they don't change)*
- Index file: An SSTable of (key, position in data sstable) pairs
- And a Bloom filter (for efficient search) – next slide

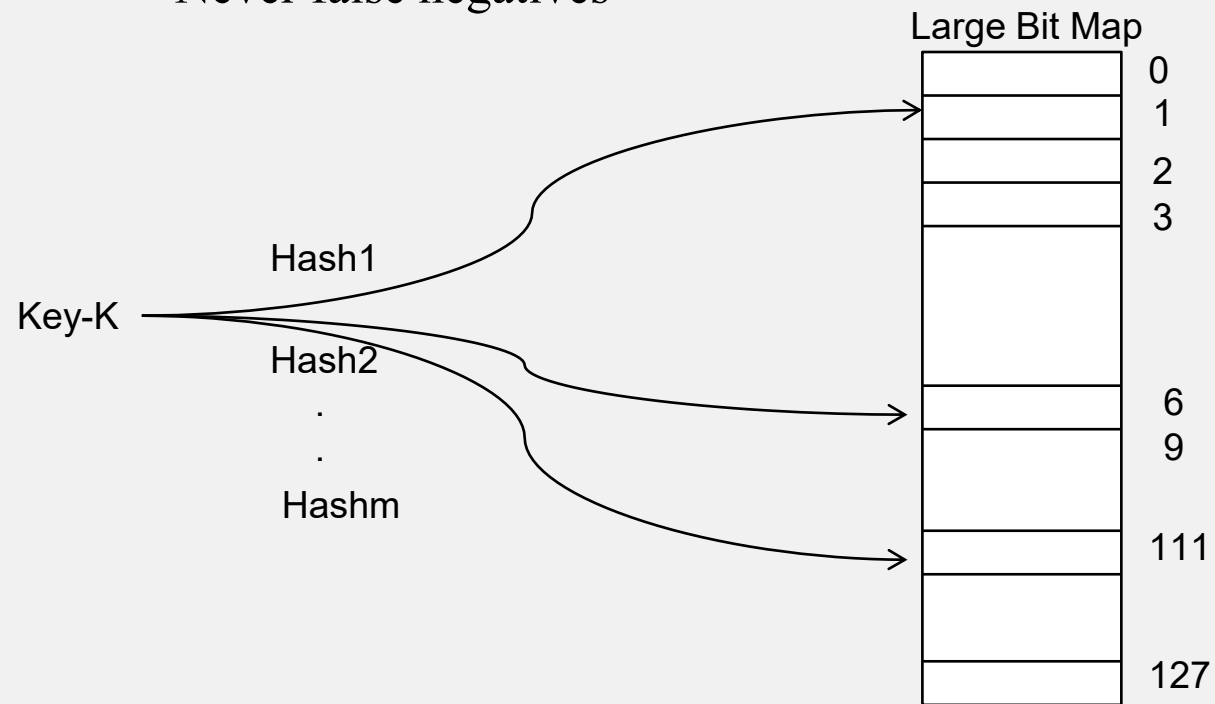
Compaction

Data updates accumulate over time and SStables and logs need to be compacted

- The process of compaction merges SStables, i.e., by merging updates for a key
- Run periodically and locally at each server

Bloom Filter

- Compact way of representing a set of items
- Checking for existence in set is cheap
- Some probability of false positives: an item not in set may check true as being in set
- Never false negatives



On insert, set all hashed bits.

On check-if-present,
return true if all hashed bits
set.

- False positives

False positive rate low

- $m=4$ hash functions
- 100 items
- 3200 bits
- FP rate = 0.02%

Reads at a Replica



Which is faster in Cassandra – reads or writes?



What if the false positive rate of Bloom filter is too high?

- Read looks at Memtables first
- And then SSTables first L0, L1, and then Ln
 - Check if SSTable key range intersects
 - If yes, check bloom filter
 - If no, repeat with the next SSTable
 - If yes, search Index to get position in SSTable
 - If no, repeat with the next SSTable
 - If yes, read actual KV pair from SSTable
- A row may be split across multiple SSTables => reads need to touch multiple SSTables => reads slower than writes (but still fast)

Deletes

Delete: don't delete item right away

- Add a **tombstone** to the log and memtable
- Eventually, when compaction encounters tombstone it will delete item

Cassandra Vs. RDBMS

- MySQL is one of the most popular (and has been for a while)
- On > 50 GB data
- MySQL
 - Writes 300 ms avg
 - Reads 350 ms avg
- Cassandra
 - Writes 0.12 ms avg
 - Reads 15 ms avg
- Orders of magnitude faster
- What's the catch? What did we lose?

Announcements

- (All) HW2 due 10/4 @ 11:59 pm (Sunday!) US Central (hard deadline)
- (All) Midterm Thursday (10/8): Written, in class
- (4cr) **MP2 Demo, Signup, and Submission instructions release on Piazza**
 - ✓ (Due by Fri) [MP Demo Signup Sheet]
 - ✓ (Due by Sun) [a. MP Submission Form & push MP code to GitHub]
 - ✓ (Due by Sun 13) [MP Report Submission on Gradescope]
 - ✓ (On Mon) MP Demos

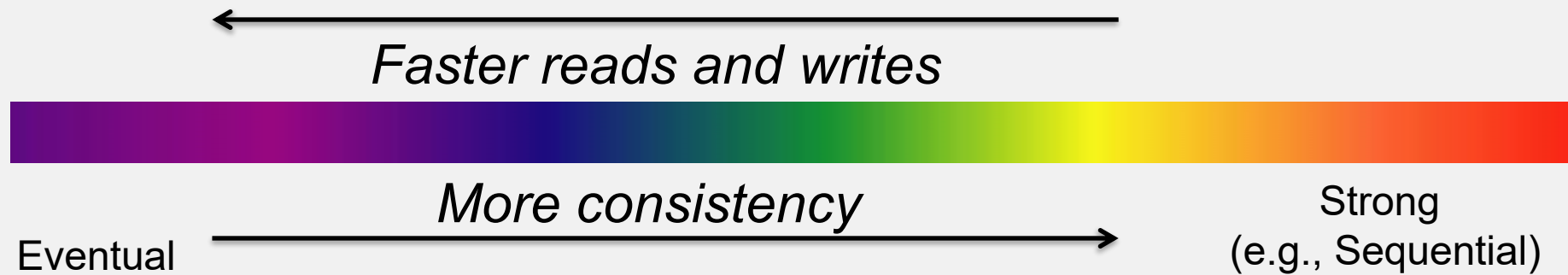
CS 425 / ECE 428
Distributed Systems
Fall 2026

Consistency Models

Types of Consistency

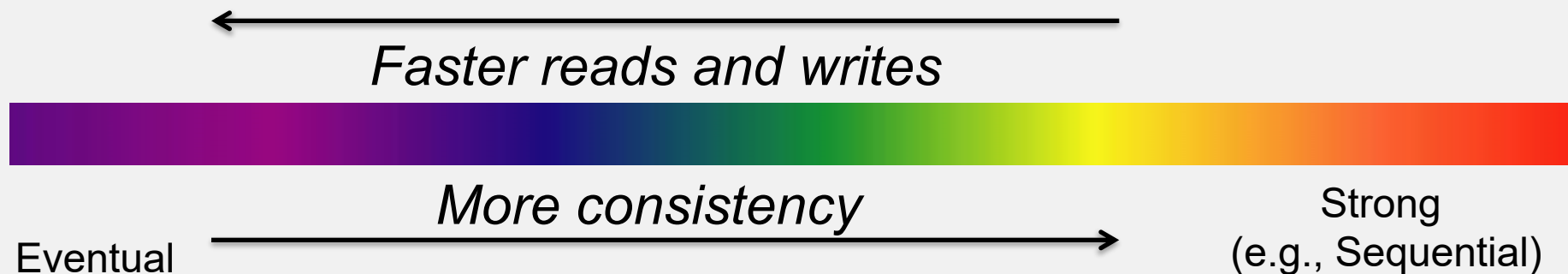
- Cassandra offers Eventual Consistency
- Are there other types of weak consistency models?

Consistency Spectrum



Spectrum Ends: Eventual Consistency

- Cassandra offers [Eventual Consistency](#)
 - If writes to a key stop, all replicas of key will converge
 - Originally from Amazon's Dynamo and LinkedIn's Voldemort systems

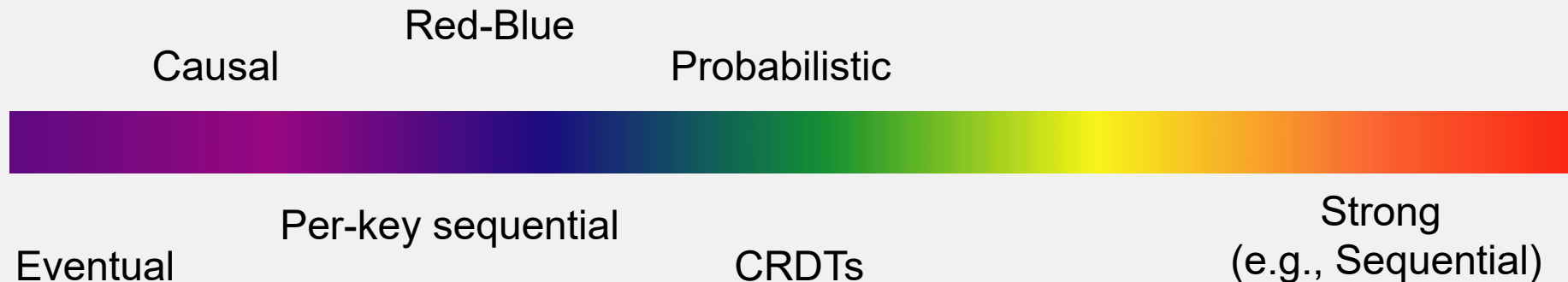


Spectrum Ends: Strong Consistency Models

- **Linearizability:** Each operation by a client is visible (or available) instantaneously to all other clients
 - Instantaneously in real time
- **Sequential Consistency** [Lamport]:
 - *"... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program."*
 - After the fact, find a “reasonable” ordering of the operations (can re-order operations) that obeys sanity (consistency) at all clients, and across clients.
- Transaction ACID properties, e.g., newer key-value/NoSQL stores (sometimes called “NewSQL”)
 - Hyperdex [Cornell]
 - Spanner [Google]
 - Yesquel [Microsoft Research], Tapir [UW], Callas [UT], Rifl [Stanford], ...

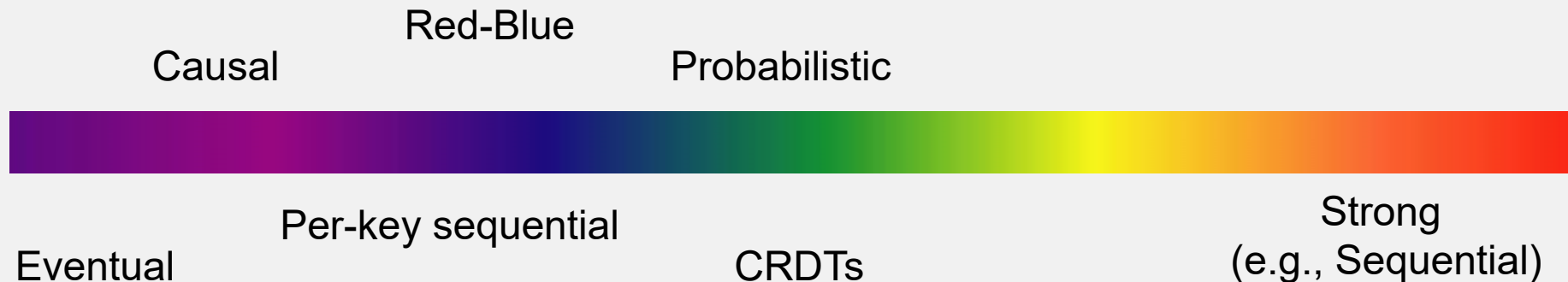
Newer Consistency Models

- Striving towards strong consistency
- While still trying to maintain high availability and partition-tolerance



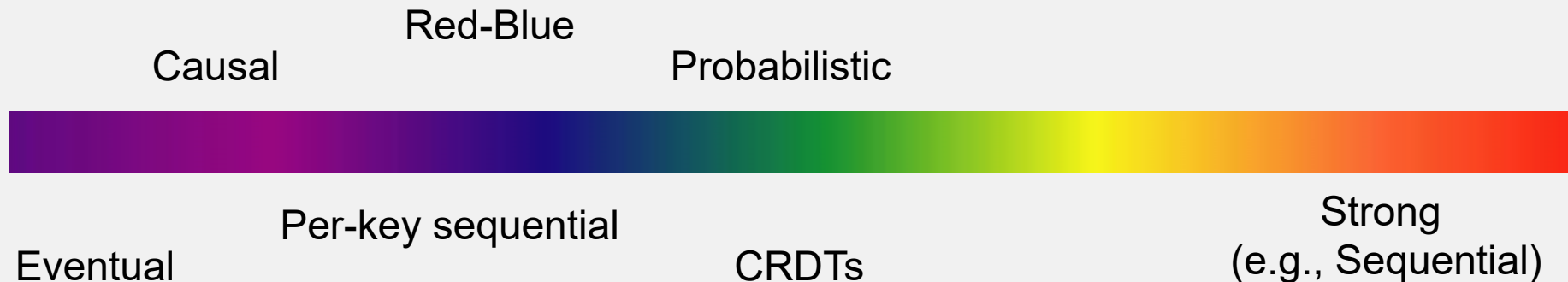
Newer Consistency Models (Contd.)

- **Per-key sequential:** Per key, all operations have a global order
- **CRDTs** (Commutative Replicated Data Types): Data structures for which commutated writes give same result [INRIA, France]
 - E.g., value == int, and only op allowed is +1
 - Effectively, servers don't need to worry about consistency



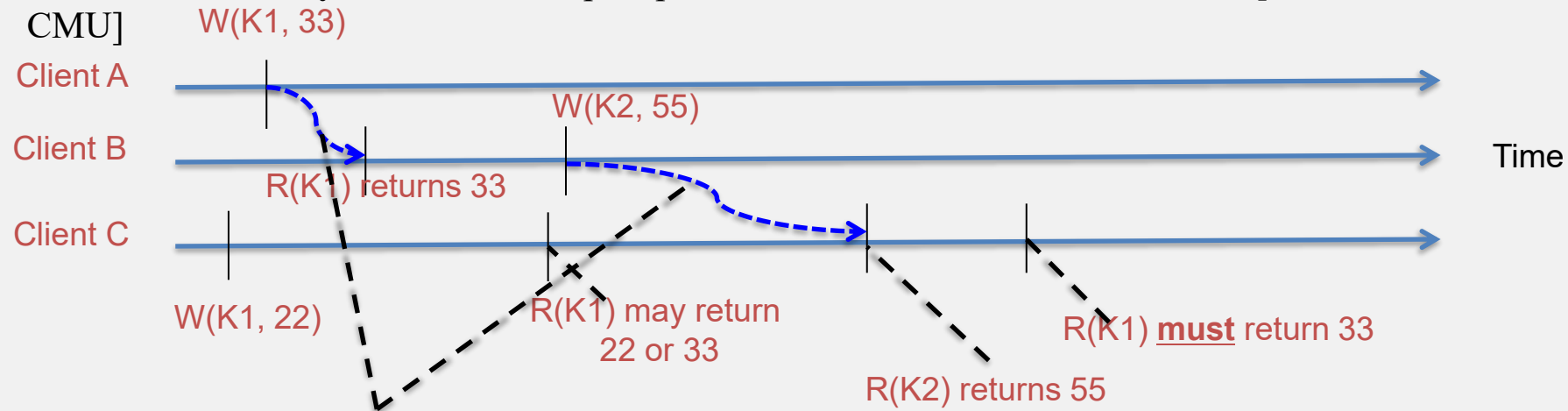
Newer Consistency Models (Contd.)

- **Red-blue Consistency:** Rewrite client transactions to separate ops into red ops vs. blue ops [MPI-SWS Germany]
 - Blue ops can be executed (commutated) in any order across DCs
 - Red ops need to be executed in the same order at each DC



Newer Consistency Models (Contd.)

Causal Consistency: Reads must respect partial order based on information flow [Princeton, CMU]



Causality, not messages

Red-Blue

Causal

Probabilistic



Eventual

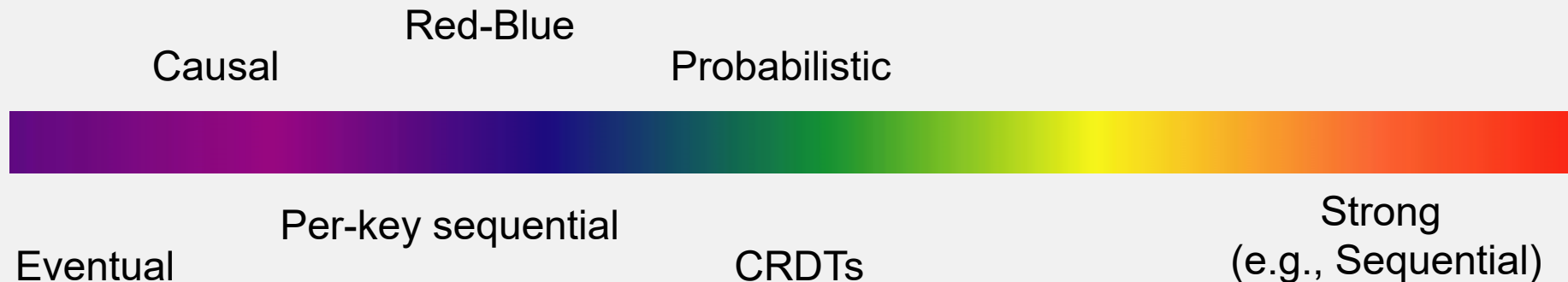
Per-key sequential

CRDTs

Strong
(e.g., Sequential)

Which Consistency Model should you use?

- Use the lowest consistency (to the left) consistency model that is “correct” for your application
 - Gets you fastest availability



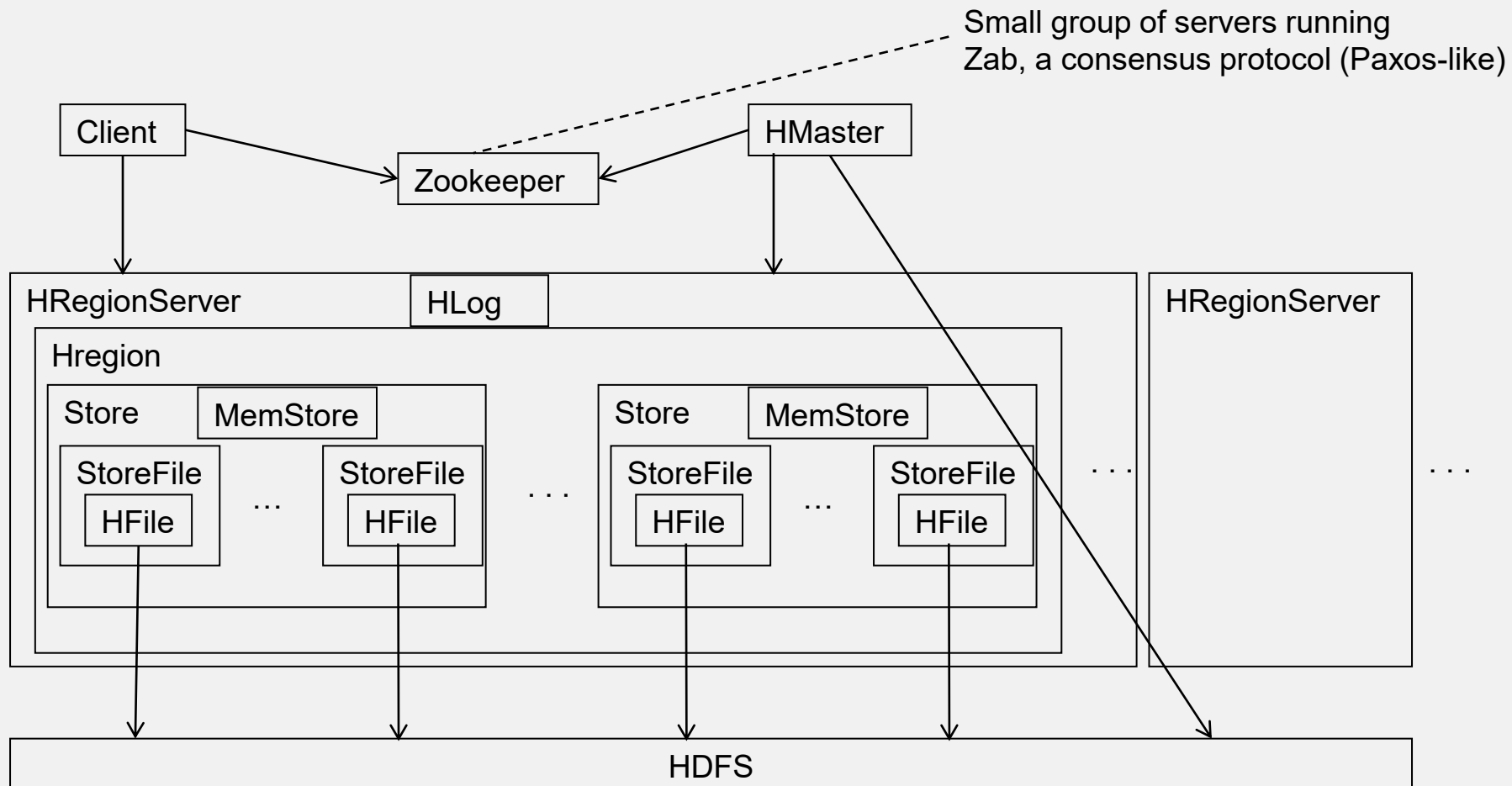
CS 425 / ECE 428
Distributed Systems
Fall 2026

HBase

HBase

- Google's BigTable was first “blob-based” storage system
- Yahoo! Open-sourced it → HBase
- Major Apache project today
- Facebook uses HBase internally
- API functions
 - Get/Put(row)
 - Scan(row range, filter) – range queries
 - MultiPut
- Unlike Cassandra, HBase prefers consistency (over availability)
- (Version of 2014)

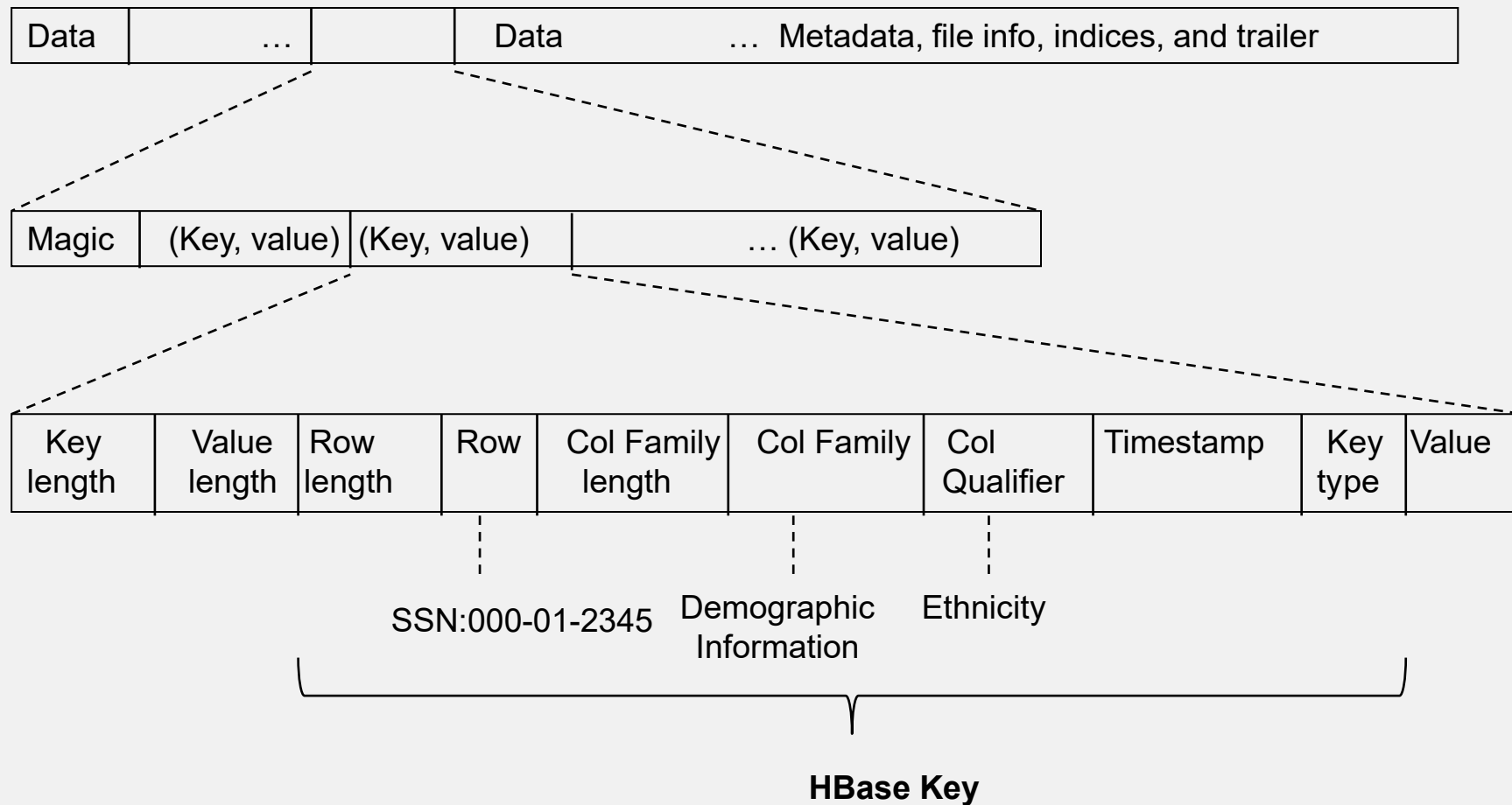
HBase Architecture



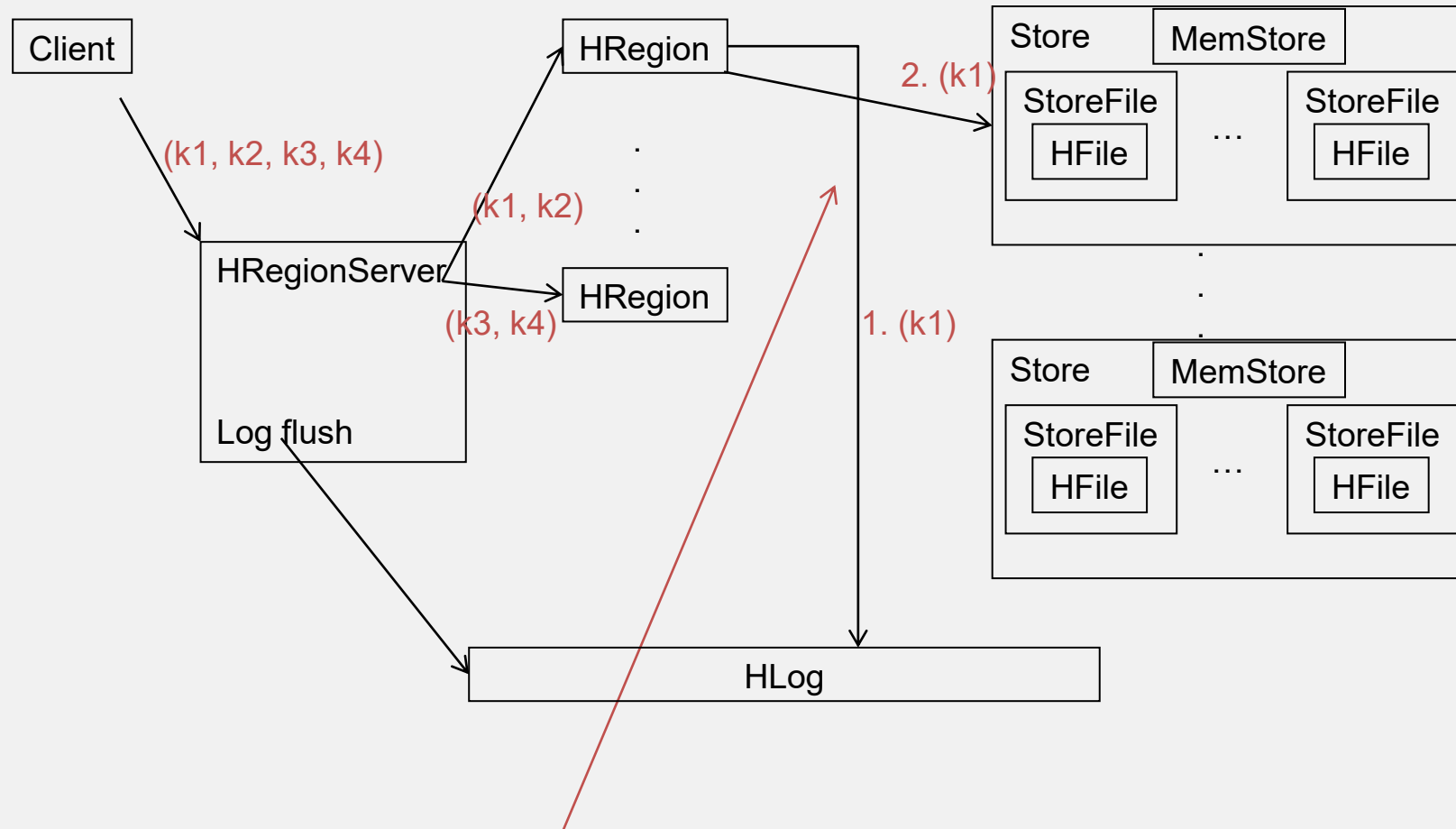
HBase Storage hierarchy

- HBase Table
 - Split it into multiple regions: replicated across servers
 - ColumnFamily = subset of columns with similar query patterns
 - One Store per combination of ColumnFamily + region
 - Memstore for each Store: in-memory updates to Store; flushed to disk when full
 - StoreFiles for each store for each region: where the data lives
 - HFile
- HFile
 - SSTable from Google's BigTable

HFile



Strong Consistency: HBase Write-Ahead Log



Write to HLog before writing to MemStore
Helps recover from failure by replaying Hlog.

Log Replay

- After recovery from failure, or upon bootup (HRegionServer/HMaster)
 - Replay any stale logs (use timestamps to find out where the database is w.r.t. the logs)
 - Replay: add edits to the MemStore

Cross-Datacenter Replication

- Single “Leader” (~~“Master”~~) cluster
- Other “Follower” (~~“Slave”~~) clusters replicate the same tables
- Leader cluster synchronously sends HLogs over to follower clusters
- Coordination among clusters is via Zookeeper
- Zookeeper can be used like a file system to store control information
 1. */hbase/replication/state*
 2. */hbase/replication/peers/<peer cluster number>*
 3. */hbase/replication/rs/<hlog>*

CS 425 / ECE 428

Distributed Systems

Fall 2026

MongoDB (additional slides)

Not in syllabus

MongoDB: A NoSQL System Installation

- MongoDB material not included in syllabus
 - <http://www.mongodb.org/downloads>
 - <http://docs.mongodb.org/manual/installation>
 - `mongod --dbpath <path-to-data>`
 - Mongo
 - (Version of 2015)
-
- (MongoDB slides adapted from Mainak Ghosh's slides)

Data Model

- Stores data in form of BSON (Binary JavaScript Object Notation) *documents*

```
{  
    name: "travis",  
    salary: 30000,  
    designation: "Computer Scientist",  
    teams: [ "front-end", "database" ]  
}
```

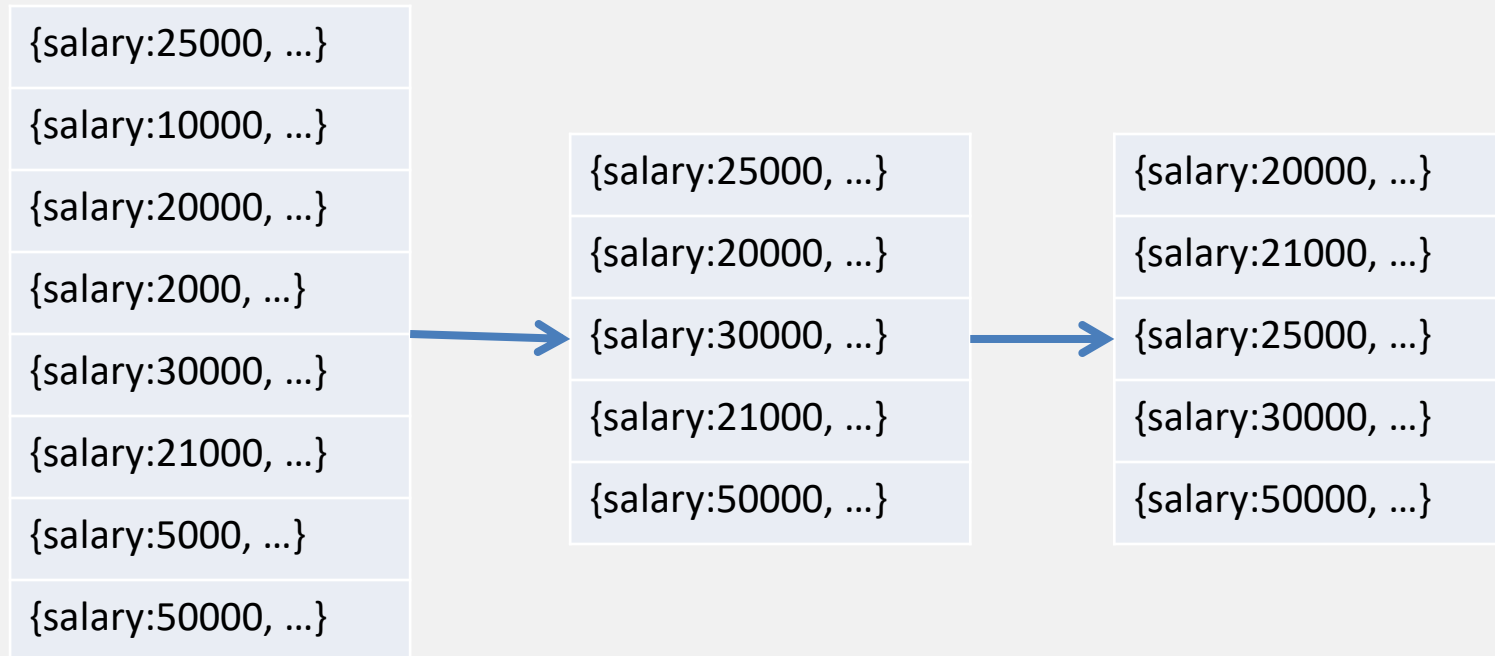
- Group of related *documents* with a shared common index is a *collection*

MongoDB: Typical Query

Query all employee names with salary greater than 18000 sorted in ascending order

```
db.employee.find( {salary:{$gt:18000}}, {name:1} ).sort( {salary:1} )
```

Collection Condition Projection Modifier



Insert

Insert a row entry for new employee Sally

```
db.employee.insert({  
    name: "sally",  
    salary: 15000,  
    designation: "MTS",  
    teams: [ "cluster-management" ]  
})
```

Update

All employees with salary greater than 18000 get a designation of Manager

<i>Update Criteria</i>	<code>{salary: {\$gt: 18000}},</code>
<i>Update Action</i>	<code>{\$set: {designation: "Manager"}},</code>
<i>Update Option</i>	<code>{multi: true}</code>
	<code>)</code>

Multi-option allows multiple document update

Delete

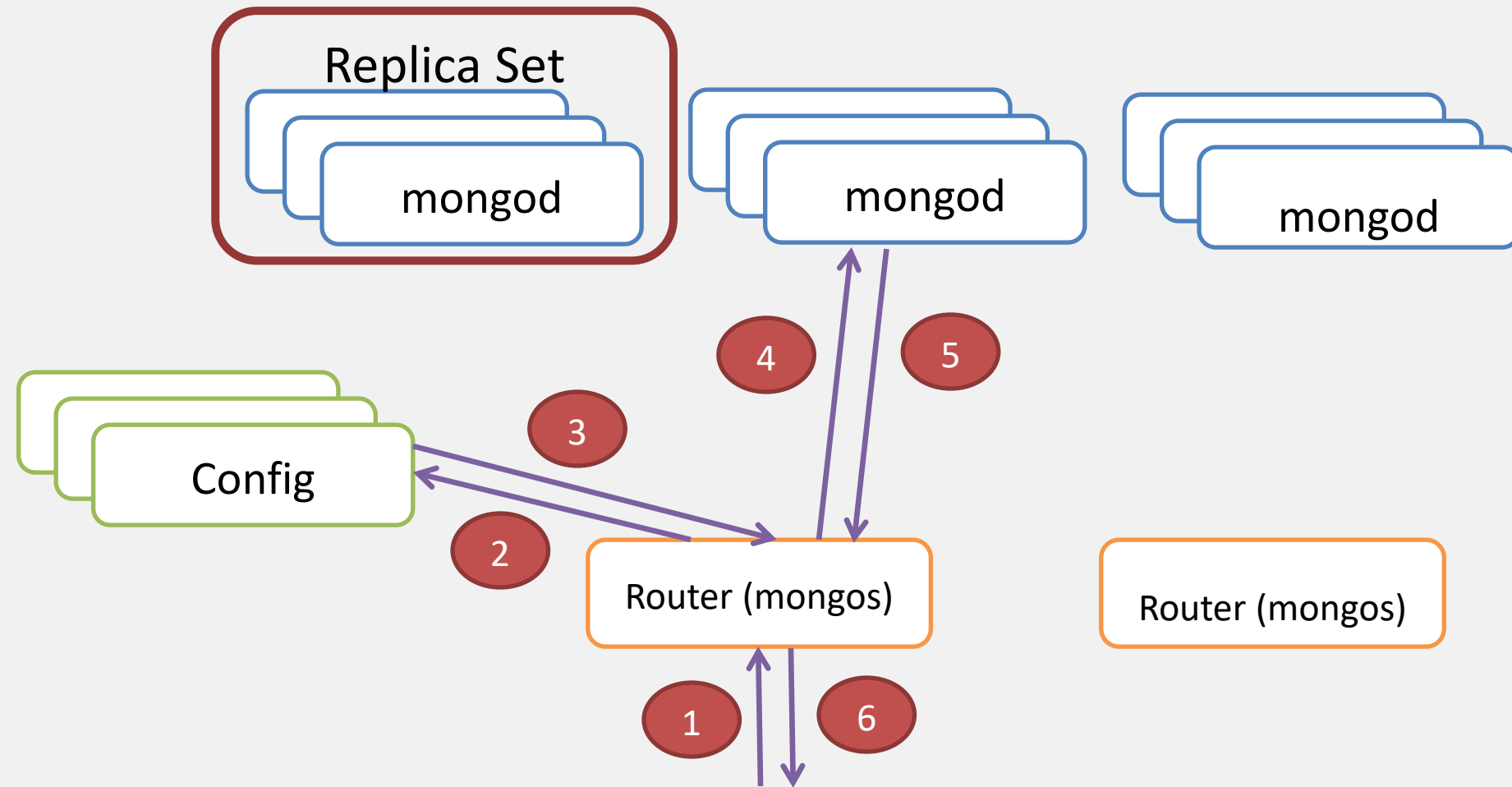
Remove all employees who earn less than 10000

Remove Criteria

```
db.employee.remove(  
  {salary:{$lt:10000}},  
)
```

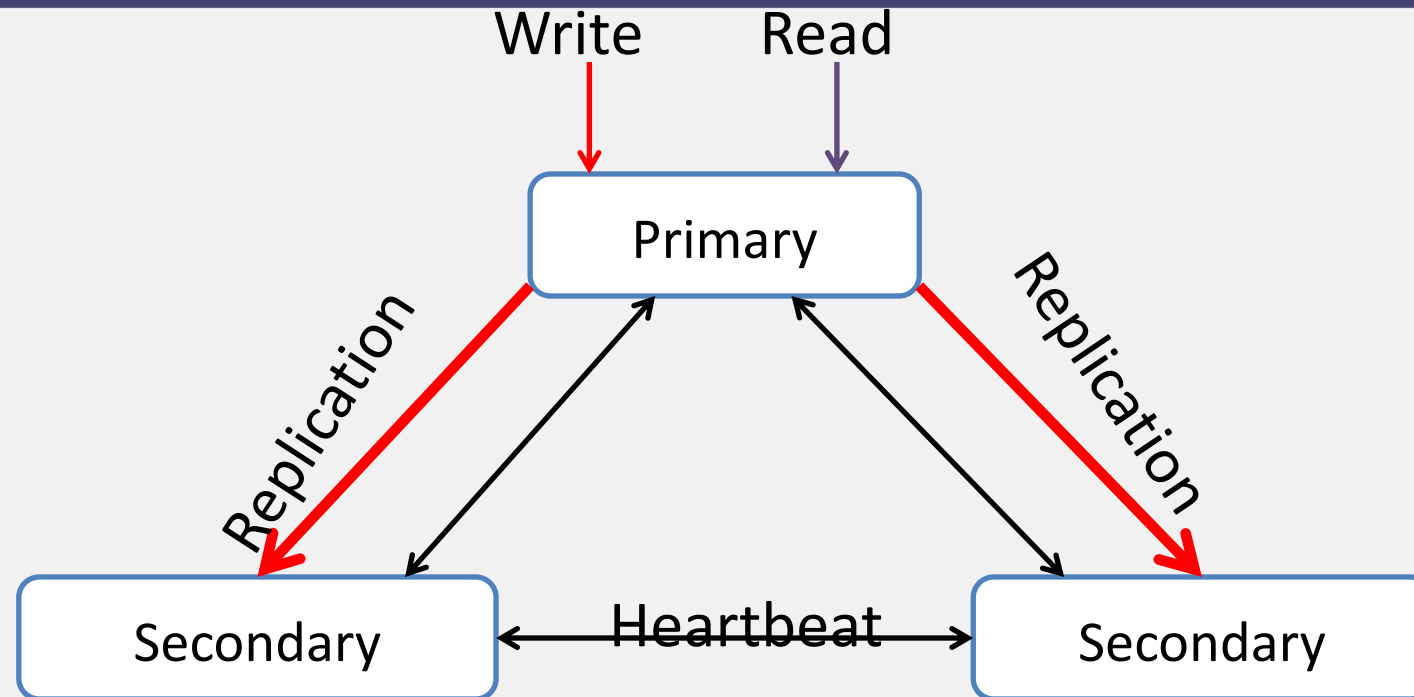
Can accept a flag to limit the number of documents removed

Typical MongoDB Deployment



- Data split into **chunks**, based on shard key (~ primary key)
 - Either use hash or range-partitioning
- **Shard**: collection of chunks
- Shard assigned to a replica set
- **Replica set** consists of multiple **mongod** servers (typically 3 mongod's)
- Replica set members are mirrors of each other
 - One is primary
 - Others are secondaries
- **Routers**: **mongos** server receives client queries and routes them to right replica set
- **Config server**: Stores collection level metadata.

Replication



Replication

- Uses an oplog (operation log) for data sync up
 - Oplog maintained at primary, delta transferred to secondary continuously/every once in a while
- When needed, Leader Election protocol elects a primary
- Some mongod servers do not maintain data but can vote – called as Arbiters

Read Preference

- (Updated 2016 v3.2)
- Determine where to route read operation
- Default is primary. Some other options are
 - primary-preferred
 - secondary
 - nearest
 - majority
- Helps reduce latency, improve throughput
- Reads from secondary may fetch stale data

Write Concern

- (Updated 2016 v3.2)
- Determines the guarantee that MongoDB provides on the success of a write operation
- 0: no ack
- 1: ack, from primary
- majority
- Weaker write concern implies faster write time

Write operation performance

- Journaling: Write-ahead logging to an on-disk journal for durability
- Journal may be memory-mapped
- Indexing: Every write needs to update every index associated with the collection

Balancing

- Over time, some chunks may get larger than others
- Splitting: Upper bound on chunk size; when hit, chunk is split
- Balancing: Migrates chunks among shards if there is an uneven distribution

Consistency

- Either strongly consistent or eventually consistent
 - Depending on combination of read preference and write concern
- CAP Theorem: With Strong consistency, under partition, MongoDB becomes write-unavailable thereby ensuring consistency

Performance

- 30 – 50x faster than MySQL Server 2008 for writes [1]
- At least 3x faster for reads [1]
- MongoDB 2.2.2 offers slower throughput for different YCSB workloads compared to Cassandra [2]

[1] <http://blog.michaelckennedy.net/2010/04/29/mongodb-vs-sql-server-2008-performance-showdown/>

[2] <http://hyperdex.org/performance/>

Summary

- Traditional Databases (RDBMSs) work with strong consistency, and offer ACID
- Modern workloads don't need such strong guarantees, but do need fast response times (availability)
- Unfortunately, CAP theorem
- Key-value/NoSQL systems offer BASE
 - Eventual consistency, and a variety of other consistency models striving towards strong consistency
- We discussed design of
 - Cassandra
 - HBase
 - MongoDB

Some more MongoDB queries

Insert

Insert a row entry for new employee Sally

use records -- Creates a database

```
db.employee.insert({  
    name: "Sally",  
    salary: 15000,  
    designation: "MTS",  
    teams: "cluster-management"  
})
```

Also can use **save** instead of **insert**



Bulk Load

```
people = ["Marc", "Bill", "George", "Eliot", "Matt", "Trey", "Tracy",  
"Greg", "Steve", "Kristina", "Katie", "Jeff"];  
money = [10000, 5000, 8000, 2000];  
position = ["MTS", "Computer Scientist", "Manager", "Director"];  
groups = ["cluster-management", "human-resource", "backend", "ui"];  
  
for(var i=0; i<10000; i++){  
    name = people[Math.floor(Math.random()*people.length)];  
    salary = money[Math.floor(Math.random()*money.length)];  
    designation =  
position[Math.floor(Math.random()*position.length)];  
    teams = groups[Math.floor(Math.random()*groups.length)];  
    db.employee.save( {name:name, salary:salary,  
designation:designation, teams:teams} );  
}
```



Query

- `db.employee.find()`
- `db.employee.find({name: "Sally"})`
- `var cursor = db.employee.find({salary: {$in: [5000, 2000] } } )`
- Use `next()` to access the rest of the records



Query

- `db.employee.find( {name: "Steve", salary: { $lt: 3000 } } )`
- `db.employee.find( { $or: [ { name: "Bill" }, { salary: { $gt: 9000 } } ] } )`
- Find records of all managers who earn more than 5000
- `db.employee.find( {designation:"Manager", salary: { $gt: 5000 } } )`



Aggregation Commands

- `db.employee.count()`
- How many employees with name Steve?
- `db.employee.find({name: "Steve"}).count()`
- `db.employee.find({name: "Steve"}).skip(10)`
- `db.employee.find({name: "Steve"}).limit(10)`



Modify

- Increment salary of all managers by 1000
- `db.employee.update( { designation : "Manager" }, { $inc : { salary : 1000 } } )`
- `db.employee.update( { designation : "Manager" }, { $inc : { salary : 1000 } } , { multi: true } )`
- Increment salary of all managers working in cluster-management group by 5000
- `db.employee.update( { designation : "Manager", teams: "cluster-management" }, { $inc : { salary : 5000 } } , { multi: true } )`



Remove

- `db.employee.remove( { name : "Sally" } )`
- Remove all Computer Scientist in the ui division
- `db.employee.remove( {teams: "ui", designation: "Computer Scientist"} )`

